

Java

(Spring Boot)

Prova

ID Alumne: 7	ID Corrector:
Cicle:	Grup:

Pregunta 1. Recorreguts sobre Strings

```
public class StringAnalysis {
    public static void main(String[] args) {
        String text = Entrada.readLine(); // INPUT
        int count = 0;
        String textLow = text.toLowerCase();

        for (int i = 0; i < textLow.length(); i++) {
            char c = textLow.charAt(i);

            if (Character.isLetter(c)){
                if(c == 'a' || c == 'e' || c == 'i' || c == 'o' || c == 'u'){
                    continue;
                }
                count++;
            }

            if (c == '.') {
                break;
            }
        }
        System.out.println("Total: " + count); //OUTPUT
    }
}
```

INPUT	OUTPUT
Anticonstitucional	
Els primers ordinadors ocupaven sales senceres. Ara caben a la butxaca.	
Tinc hora al Dr. Pérez. Arribaré tard.	

Pregunta 2. Validació d'Strings

Implementar un programa en Java que validi si una data introduïda per l'usuari segueix el format **DD/MM/AAAA**.

Nota: La part de l'enunciat subratllat és opcional.

Les condicions per considerar que la data és vàlida són les següents:

1. **Dia (DD):** Els dos primers caràcters han de ser dígit i han de representar un valor numèric entre 01 i 31.
2. **Mes (MM):** Els dos caràcters següents han de ser dígit i han de representar un valor numèric entre 01 i 12.
3. **Any (AAAA):** Els quatre últims caràcters han de ser dígit i han de representar un valor numèric de quatre dígit (entre 1000 i 9999).
4. **Barres ("/"):** La data ha de contenir exactament dues barres ("/"), una després del dia i una altra després del mes (per exemple, en la data DD/MM/AAAA, les barres han de ser a les posicions 3 i 6).
5. La longitud total de la cadena ha de ser **10 caràcters**.

```
public class ValidarData {  
  
    public static void main(String[] args) {  
  
        System.out.println("Introdueix una data per validar (format DD/MM/AAAA): ");  
        String data = Entrada.readLine();  
  
        //El teu codi AQUÍ  
  
    }  
}
```

ID Alumne: 7	ID Corrector:
Cicle:	Grup:

Pregunta 3. Bucles dins de bucles + Recorreguts sobre Strings

```

public class StringModifier {
    public static void main(String[] args) {
        String input = Entrada.readLine(); // INPUT
        String letters = "etoia";
        String numbers = "37014";
        String modifiedString = "";
        boolean modified;

        for (int i = 0; i < input.length(); i++) {
            modified = false;
            char currentChar = input.charAt(i);

            for (int j = 0; j < letters.length(); j++) {
                if (Character.toLowerCase(currentChar)==letters.charAt(j)) {
                    modifiedString += numbers.charAt(j);
                    modified = true;
                    break;
                }
            }

            if(!modified){
                modifiedString += currentChar;
            }
        }
        System.out.println(modifiedString); //OUTPUT
    }
}

```

INPUT	OUTPUT
Automàtic	
El caixer automàtic no funciona correctament	
Mussols	

Programación Orientada a Objetos

Clases y objetos

- Propiedades
 - Propiedades mutables
 - Propiedades publicas
 - Propiedades protegidas
- Acceso externo
- Preguntas a un objeto
- Acciones de un objeto

Constructores

Miembros de instancias y de clases

Composición y agregación

Herencia

- Concepto de herencia
- Interfaz de una clase
- Comportamientos específicos
- Herencia y constructores
- Polimorfismo
- Clases abstractas
- Peligro de abuso de herencia
- interfaces

Map

Map es una estructura clave-valor que se implementa comúnmente usando la interfaz Map del paquete java.util de Java. La implementación más utilizada es HashMap.

Ejemplo de un HashMap:

```
import java.util.HashMap;
```

```
import java.util.Map;
```

```
public class EjemploMap {
```

```
    public static void main(String[] args) {
```

```
        Map<String, Integer> edades = new HashMap<>();
```

```
        // Agregar pares clave-valor
```

```
        edades.put("Jeiron", 22);
```

```
        edades.put("Junior", 23);
```

```
        edades.put("JeyJey", 20);
```

```
        int edadJunior = edad.get("Junior");
```

```
        System.out.println("Edad de Junior: " +  
edadJunior);
```

```
        // Recorrer el Map
```

```
        for (String nombre: edades.keySet()) {
```

```
            System.out.println(nombre + " tiene " +  
edades.get(nombre) + " años");
```

```

// Verificar si una clave existe
if (edades.containsKey("JeyJey")) {
    System.out.println("JeyJey está en el
mapa");
}
}

// Eliminar un elemento
edades.remove("Jeiron");
}
}

```

Tener en cuenta que:

- HashMap es rápido y no garantiza orden.
- TreeMap ordena automáticamente las claves.
- LinkedHashMap mantiene el orden de inserción.

Explicación de las 3 implementaciones:

1. HashMap:

Orden: no garantiza ningún orden en las claves.

Velocidad: muy eficiente para operaciones como insertar, buscar y eliminar (complejidad promedio $O(1)$).

Permite claves null: sí, permite una clave null.

Ejemplo:

```
Map<String, Integer> mapa = new HashMap<>();  
mapa.put("B", 2);  
mapa.put("A", 1);  
mapa.put("C", 3);
```

System.out.println(mapa); // El orden puede ser A-B-C o cualquier otro.

2. TreeMap:

Orden: ordena automáticamente las claves de menor a mayor (según orden natural o comparador).

Velocidad: un poco más lento ($O(\log n)$) porque mantiene orden.

Permite claves null: no, lanza NullPointerException.

Ejemplo:

```
Map<String, Integer> mapa = new TreeMap<>();  
mapa.put("B", 2);  
mapa.put("A", 1);  
mapa.put("C", 3);
```

System.out.println(mapa); // Siempre imprime: {A=1, B=2, C=3}

3. LinkedHashMap:

Orden: mantiene el orden de inserción.

Velocidad: similar a HashMap, pero con un poco más de consumo por mantener el orden.

Permite clave null: sí.

Ejemplo:

```
Map<String, Integer> mapa = new TreeMap<>();
```

```
mapa.put("B", 2);
```

```
mapa.put("A", 1);
```

```
mapa.put("C", 3);
```

```
System.out.println(mapa); // Siempre imprime: {B=2,  
A=1, C=3}
```

Cómo funcionan internamente y cuándo usar cada uno

1. HashMap

¿Cómo funciona?

- Usa una tabla hash para almacenar los pares clave-valor.
- Java convierte la clave en un número (hash) y lo usa para decidir dónde guardar el valor.
- Si varias claves tienen el mismo hash, se usa una lista o árbol para resolver colisiones.

¿Cuándo usarlo?

- Cuando no te importe el orden de los elementos.
- Cuando necesitas buscar datos rápidamente por una clave.

Ejemplo:

```
HashMap<Integer, String> usuarios = new HashMap<>();  
usuarios.put(1, "Jeiron");  
usuarios.put(2, "Junior");
```

```
System.out.println(usuario.get(2)); // Junior
```

2. TreeMap

¿Cómo funciona?

- Usa un árbol rojo-negro (estructura ordenada).
- Las claves se ordenan automáticamente de menor a mayor (o con un comparador).
- Cada vez que añades una clave, el árbol se reorganiza.

¿Cuándo usarlo?

- Cuando necesitas ordenar los datos por clave.
- Cuando quieres recorrer los elementos ordenadamente.

Ejemplo:

```
TreeMap<String, Double> notas = new TreeMap<>();  
notas.put("Jeiron", 8.5);  
notas.put("Junior", 7.0);  
notas.put("JeyJey", 9.0);
```

```
// Imprimir en orden: Jeiron, JeyJey, Junior
```

```
for (String nombre: notas.keySet()) {  
    System.out.println(nombre + ": " +  
        notas.get(nombre));  
}
```

3. LinkedHashMap:

¿Cómo funciona?

- Es como un HashMap, pero mantiene una lista doblemente enlazada con el orden de inserción.
- Es un poco más lento que HashMap, pero mantiene el orden.

¿Cuándo usarlo?

- Cuando quieres preservar el orden en que insertaste los datos.
- Cuando necesitas tanto velocidad como orden.

Ejemplo:

```
LinkedHashMap<Integer, String> pasos = new  
    LinkedHashMap <>();
```

```
pasos.put(1, "Encender el horno");
```

```
pasos.put(2, "Mezclar ingredientes");
```

```
pasos.put(3, "Hornear 30 minutos");
```

```
// Imprimir en orden: Jeiron, JeyJey, Junior
```

```
for (String paso: pasos.values()) {
```

```
    System.out.println(paso);
```

```
}
```

Tabla de hash Java

Una tabla hash (o hash table) es una estructura de datos, que almacena pares clave-valor y permite acceder a los valores de forma muy eficiente utilizando la clave. Internamente, usa una función hash para calcular el índice donde almacenar o buscar cada elemento.

Elementos:

- Clave (key): es única, se usa para identificar un valor.
- Valor (value): es el dato asociado a la clave.
- Función hash: convierte la clave en un número (hash code) que se usa como índice en un array.
- Colisiones: ocurren cuando dos claves distintas generan el mismo índice; se resuelven con técnicas como listas enlazadas o direccionamiento abierto.

Ejemplos:

Hay dos clases principales que implementan tablas hash en Java:

1. `HashMap<K, V>` en `java.util`.

- a. Permite claves y valores null.
- b. No está sincronizada (no es segura para hilos).
- c. Orden no garantizado.

```
import java.util.HashMap;
```

```
public class Tabla {  
    public static void main(String[] args) {  
        HashMap<String, Integer> edades = new HashMap<>();  
        edades.put("Jeiron", 22);  
        edades.put("Junior", 23);  
  
        System.out.println(edades.get("Jeiron")); // imprime  
        22  
    }  
}
```

2. `Hashtable<K, V>` en `java.util`.

- a. Es más antigua.
- b. Está sincronizada (segura para hilos).
- c. No permite claves ni valores null.

Método hashCode()

El método hashCode se puede sobrescribir en Java y puedes hacer que siempre devuelva 1, lo que está cumpliendo técnicamente con el contrato de hashCode, pero introduce un problema serio de rendimiento

El contrato de hashCode() dice que:

1. Si dos objetos son iguales según equals(), deben tener el mismo hashCode().
2. Si dos objetos tienen el mismo hashCode(), no necesariamente son iguales.

Por lo que devolver siempre 1 no rompe el contrato, pero sí hace que todas las instancias tengan el mismo hash, lo que afecta el rendimiento de las estructuras como HashMap, HashSet o Hashtable.

Consecuencias de devolver siempre 1:

- Todos los objetos van a la misma posición en la tabla hash (misma "bucket").
- Se pierde el beneficio de la búsqueda rápida ($O(1)$) que da una buena dispersión de hashes.
- Las operaciones en HashMap pasan de tiempo constante a tiempo lineal ($O(n)$).

A veces se usa return 1; como implementación temporal durante el desarrollo o pruebas, pero nunca debería dejarse en producción.

Gestionarlo correctamente:

1. Evitar dejarlo así en producción.
2. Usar los campos relevantes del objeto para calcular el hash de forma eficiente:

```
@Override
public int hashCode() {
    return Objects.hash(campo1, campo2, campo3);
}
```

o manualmente:

```
@Override
public int hashCode() {
    int result = 17;
    result = 31 * result + (campo1 != null ?
campo1.hashCode() : 0);
    result = 31 * result + campo2;
    return result;
}
```